Тема 04. ООП и первые паттерны проектирования

Блок 1 · Предыдущий: 03 · Следующий: 05. Тестирование

> **Время:** 8–12 часов на два шага — 04.1 (5–7 ч), 04.2 (3–5 ч).
> Обоснование — TIME.md. Незнакомые слова — словарь.

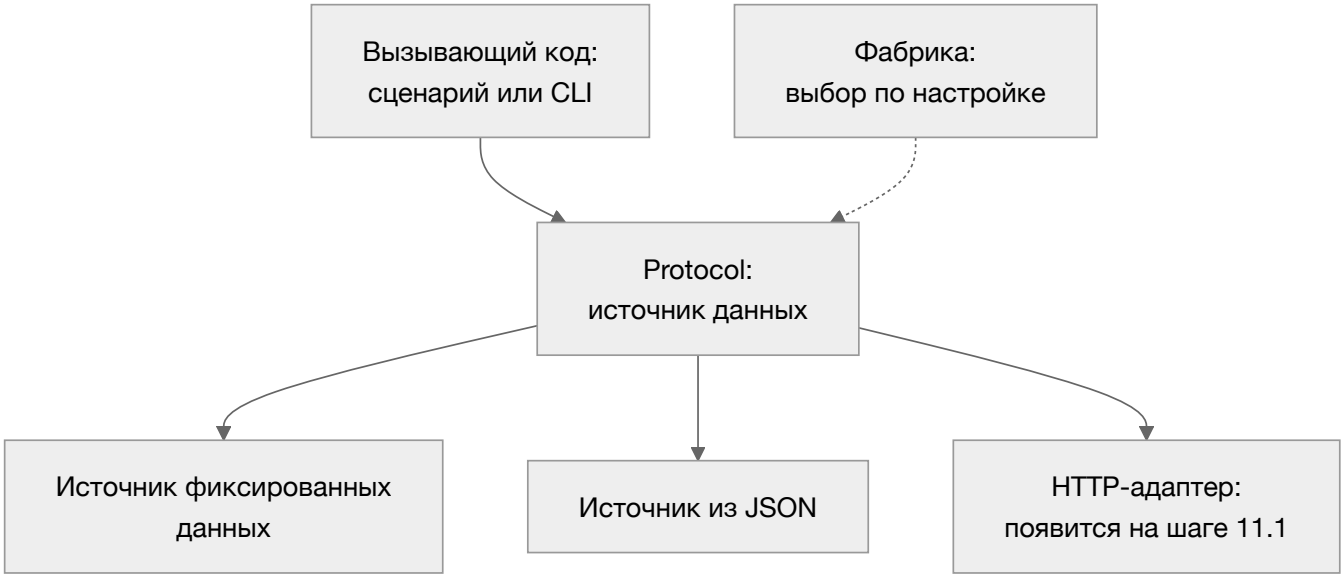
Зачем эта тема

Здесь появляется первое архитектурное решение: код перестаёт зависеть от конкретного источника данных. Благодаря этому на шаге 05.1 можно будет тестировать логику без сети, а на шаге 11.1 — подключить настоящий внешний сервис, не переписывая приложение.

Теория

- **Композиция вместо наследования** — объект получает нужное поведение через зависимость, а не через родительский класс. Наследование связывает жёстко, композиция — гибко.
- **Protocol** — описание требуемого поведения без наследования: любой класс с подходящими методами подходит автоматически, и mypy это проверяет. Такой подход называют **структурной типизацией**: «умеет то, что нужно — значит подходит». Питоничная замена абстрактным классам.
- **Strategy** — взаимозаменяемые реализации одной задачи, выбираемые во время работы программы. Протокол плюс две реализации под ним — это уже Strategy, отдельного «паттерна» писать не нужно.
- **Простая Factory** — функция выбора реализации по конфигурации. Это не обязательно GoF Factory Method: у него создание делегируется переопределяемому методу.
- **Adapter** — обёртка, приводящая чужой интерфейс к вашему. Ключевой приём изоляции внешних систем от вашего кода; применять его будете на шаге 11.1.
- **Паттерн в Python часто проще** — где в других языках нужен класс, здесь хватает функции или декоратора. Паттерн — это решение проблемы, а не обязательный ритуал.
- **State** — поведение зависит от состояния. Для нашего масштаба достаточно `Enum`, таблицы разрешённых переходов и доменного исключения при запрете.

Что получится на шаге 04.1:



Вызывающий код знает только протокол — прямоугольник в центре. Он не знает имён классов снизу и не проверяет их тип. Поэтому на шаге 11.1 можно будет подставить настоящий HTTP-адаптер, не тронув ни строчки в сценарии.

Наследование и композиция

Наследование выражает «является»: подкласс получает весь код родителя и зависит от его внутреннего устройства. Правка родителя может незаметно сломать всех наследников, а комбинации поведения («кешированный источник из JSON с повторами») порождают взрыв подклассов. Композиция выражает «содержит»: объект получает помощника через конструктор и пользуется только его публичными методами.

```
class CachedSource:
    """Кеширует ответы любого источника — не только JSON."""

    def __init__(self, inner: VacancySource) -> None:
        self._inner = inner
        self._cache: dict[str, list[Vacancy]] = {}

    def fetch(self, query: str) -> list[Vacancy]:
        if query not in self._cache:
            self._cache[query] = self._inner.fetch(query)
        return self._cache[query]
```

Protocol и абстрактный класс

```
from abc import ABC, abstractmethod
from typing import Protocol

class VacancySource(Protocol):          # структурная типизация
    def fetch(self, query: str) -> list[Vacancy]: ...

class FixedSource:                      # протокол не наследуется
    def fetch(self, query: str) -> list[Vacancy]:
        return [Vacancy(title="Python-разработчик")]

def collect(source: VacancySource, query: str) -> list[Vacancy]:
    return source.fetch(query)          # муру сверит сигнатуру fetch

class BaseSource(ABC):                  # номинальная типизация
    @abstractmethod
    def fetch(self, query: str) -> list[Vacancy]: ...
```

Абстрактный класс требует явного наследования и проверяет реализацию при создании объекта: подкласс без `fetch` упадёт с `TypeError`. В него можно положить общий код для всех наследников. Протокол не требует от реализации ничего, кроме подходящих методов, и проверяется mypy до запуска; поэтому под него подходят и чужие классы, которые вы не можете изменить. `@runtime_checkable` разрешает `isinstance` с протоколом, но проверяет только наличие методов, а не их сигнатуры.

Strategy и фабрика

Стратегия в Python часто не требует класса: если у поведения один метод, достаточно функции, а тип стратегии — `Callable[[Vacancy], bool]`. Класс нужен, когда у стратегии есть состояние или несколько связанных методов. Фабрика собирает выбор реализации в одном месте:

```
def make_source(settings: Settings) -> VacancySource:
    match settings.source:
        case "fixed":
            return FixedSource()
        case "json":
            return JsonFileSource(settings.data_path)
    raise ValueError(f"неизвестный источник: {settings.source!r}")
```

Остальной код получает готовый объект и не знает, какой класс выбран. Добавление третьей реализации меняет фабрику и не меняет вызывающий код. GoF Factory Method устроен иначе: базовый класс вызывает переопределяемый метод создания, а подклассы решают, что создать.

State через таблицу переходов

```
class TransitionError(Exception):
    """Переход между этими состояниями запрещён правилами домена."""

ALLOWED: dict[Status, frozenset[Status]] = {
    Status.SAVED: frozenset({Status.APPLIED}),
    Status.APPLIED: frozenset({Status.INTERVIEW, Status.REJECTED}),
    Status.REJECTED: frozenset(),        # конечное: переходов наружу нет
}

def move_to(self, new: Status) -> None:  # метод сущности Application
    if new == self.status:
        return                           # повтор ничего не меняет
    if new not in ALLOWED[self.status]:
        raise TransitionError(f"{self.status} -> {new}")
    self.status = new
```

Проверка идёт **до** изменения, поэтому при отказе объект остаётся прежним. Правила лежат данными, а не цепочкой `if`: добавить переход — значит дописать элемент таблицы.

Принципы проектирования

- **SRP** — у модуля одна причина меняться: разбор JSON и правило приоритета живут отдельно.
- **OCP** — новый источник добавляется новым классом, а не правкой работающего сценария.
- **LSP** — реализация протокола подставляется без сюрпризов: fake не возвращает `None` там, где настоящий источник возвращает список.
- **ISP** — узкие интерфейсы: коду, который вызывает два метода, не нужны двадцать.
- **DIP** — сценарий зависит от протокола, а конкретную реализацию ему передают снаружи.
- **DRY** — одно знание в одном месте; **KISS** — простейшее работающее решение; **YAGNI** — не писать то, что понадобится «когда-нибудь».

Singleton — объект в единственном экземпляре, доступный отовсюду. В Python его роль играет любой объект на уровне модуля. Беда в скрытом общем состоянии: тест, изменивший глобальный объект, влияет на следующие тесты, а подменить его можно только патчем. Передача зависимости аргументом решает ту же задачу явно.

Материалы

- Паттерны проектирования на Python — на русском; читать Стратегию, Фабричный метод, Адаптер.
- typing.Protocol — синтаксис и примеры.
- PEP 544: Protocols — идея структурной типизации, разделы Introduction и Definitions. Читать после документации, а не вместо неё.
- abc: абстрактные базовые классы — альтернатива протоколам.

Задание

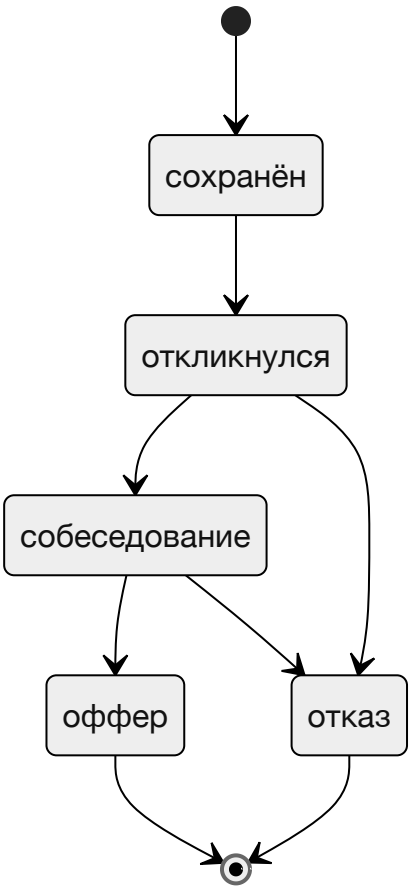
Шаг 04.1. Взаимозаменяемые источники данных (5–7 ч)

1. Определите `Protocol` для источника данных вашего проекта (поставщик вакансий, проверяльщик адресов, каталог книг, загрузчик лент, проверяльщик ссылок — в зависимости от варианта).
2. Сделайте две работающие реализации: источник фиксированных данных и источник из локального JSON подготовки. Настоящий HTTP-адаптер заменит второй на шаге 11.1.
3. Реализуйте выбор реализации по конфигурации — это и есть Factory.
4. Убедитесь, что вызывающий код нигде не проверяет тип объекта и не знает имён классов.
5. В README или в докстринге модуля в двух-трёх предложениях объясните принятое решение.
6. **По желанию:** добавьте вторую точку вариативности — способ дедупликации, правило приоритета или формат вывода. Это углубляет тему, но без него шаг считается закрытым: взаимозаменяемые источники из пункта 2 уже дают практику Strategy.

Шаг 04.2. Состояния и правила переходов (3–5 ч)

1. Реализуйте правило состояния из паспорта. Опишите разрешённые переходы таблицей: ключ — текущее состояние, значение — список допустимых следующих.
2. Выполните успешный и запрещённый переход. После отказа объект не должен измениться, а вызывающий код должен получить доменное исключение, а не `ValueError` из глубины.
3. Отдельно решите, что делает переход в то же состояние, в котором объект уже находится. В этой программе такой переход считается допустимым и ничего не меняет — это называется идемпотентностью и пригодится на шаге 13.3. Запрещать надо переходы, ломающие смысл (например, выход из конечного состояния), а не повторное выполнение того же действия.

Пример таблицы переходов для варианта А (свой вариант смотрите в паспорте):



Стрелка есть — переход разрешён. Стрелки нет — запрещён, и объект остаётся как был. Из «оффера» и «отказа» стрелок наружу нет: это конечные состояния. Повторный перевод в то же состояние стрелкой не рисуют, но он разрешён и ничего не меняет.

4. **По желанию:** объясните SRP и DIP на своём коде. SRP (single responsibility) — у класса одна причина меняться; DIP (dependency inversion) — код зависит от протокола, а не от конкретной реализации. Приведите один пример DRY, KISS или YAGNI. Прочитайте про Singleton и объясните, чем глобальный объект мешает независимым тестам.

Обязательные критерии

- ▢ Реализация источника подменяется через конфигурацию без правки вызывающего кода.
- ▢ В коде нет ветвлений вида `isinstance(...)` для выбора поведения.
- ▢ mypy подтверждает соответствие обеих реализаций протоколу.
- ▢ Обе реализации источника возвращают данные; выбор не приводит к «не реализовано».
- ▢ Есть таблица переходов; запрещённая операция даёт доменную ошибку без изменения состояния.
- ▢ Поведение перехода в уже установленное состояние выбрано осознанно и описано.

Желательные критерии

- ▢ Протокол описывает поведение в терминах вашего домена, а не чужого API.
- ▢ Добавлена вторая точка вариативности (дедупликация, приоритет или формат вывода).
- ▢ Ученик объясняет, какую проблему решает каждый применённый паттерн.

- ▢ Объяснены SRP и DIP на своём коде, приведён пример DRY, KISS или YAGNI.
- ▢ Объяснено, чем глобальный объект (Singleton) мешает независимым тестам.

На что обратить внимание

- Протокол должен быть узким: чем меньше в нём методов, тем проще реализации и тесты.
- Не создавайте абстракцию «на будущее»: две реальные реализации — достаточное основание, одна — нет.
- Фабрика возвращает тип протокола, а не конкретный класс.
- Не пытайтесь применить здесь все паттерны из справочника. Обязательного минимума — протокол, две реализации, фабрика и таблица переходов — достаточно для всей программы.

Вопросы для собеседования

Проверьте себя по темам этого шага. Ответов в программе нет намеренно: формулировка своими словами и есть проверка. Где можно — отвечайте на примере своего кода.

1. Чем композиция отличается от наследования и почему её чаще предпочитают?
2. Что такое `Protocol` и чем структурная типизация отличается от наследования?
3. Когда вы выберете `abc.ABC`, а когда `Protocol`?
4. Что такое Strategy? Является ли протокол с двумя реализациями уже Strategy?
5. Чем простая фабрика отличается от GoF Factory Method?
6. Что такое Adapter и какую проблему он решает на границе с чужой системой?
7. Почему ветвление по `isinstance` для выбора поведения — плохой признак?
8. Почему протокол должен быть узким? Что ломается, если в нём двадцать методов?
9. Как выразить State без отдельного класса на каждое состояние?
10. Что должно произойти при запрещённом переходе и в каком состоянии останется объект?
11. Разрешён ли переход в состояние, в котором объект уже находится? Как это связано с идемпотентностью?
12. Объясните SRP и DIP на примере своего кода.
13. Чем глобальный объект (Singleton) мешает независимым тестам?
14. Сколько реальных реализаций нужно, чтобы абстракция была оправдана?

Полный набор по блоку — вопросы по блоку 1.